

When Does Memory Help? A Cost-Aware Evaluation of Long-Term Memory in Tool-Using LLM Agents

Shweta Mishra
Independent Research
1196shweta@gmail.com

July 2026 — preprint v0.9*

Abstract

Long-term memory is widely assumed to improve the performance of LLM-based agents, and a rapidly growing ecosystem of memory systems — retrieval-augmented stores, rolling summarization, and structured fact memories — now competes on conversational recall benchmarks such as LoCoMo and LongMemEval. These benchmarks measure *question answering over dialogue history*, not the setting where memory is most consequential in production: **tool-using agents executing multi-step tasks across sessions**. We present MERIT (Memory Evaluation for Realistic Instrumented Tasks), a benchmark and evaluation harness that measures the *marginal utility* of memory for task-executing agents under explicit cost accounting. MERIT provides (i) episodic tool-use tasks in three domains where a controllable fraction of episodes depend on facts established in earlier episodes, verified non-re-derivable by an automated leak check; (ii) a three-tier **difficulty ladder** — single-fact, multi-fact, and *updated-fact* recall; (iii) controlled **memory corruption** (stale, contradiction, distractor); and (iv) full token and dollar accounting for every memory operation. In a pilot study (6 memory conditions $\times$ 3 domains $\times$ 3 difficulty tiers, 5,440 scored episodes, gpt-4.1-mini), memory conditions lift dependent-task success from a leak-verified floor of 0.00 (no memory) to 0.90–1.00 on single-fact tasks (all Holm-adjusted $p \leq 0.001$). The tiers dissociate architectures sharply: when a remembered fact is *updated* mid-arc, retrieval memory collapses (success 0.20–0.60; it acts on the correct value only 47% of the times that value is retrieved), while a structured fact store with update-on-write semantics stays at 0.90–1.00 — yet the *hybrid* of the two is worse than the fact store alone (0.45–0.75). Under explicit cost accounting, the most accurate condition (full replay) is the least economical: the structured store delivers equal or better dependent-task success at $\sim 14\times$ higher marginal utility per dollar. We release the benchmark, harness, and all traces for reproducible, cost-aware comparison of agent memory systems.

1 Introduction

LLM-based agents that plan, call tools, and act over multiple sessions are moving rapidly into production — customer support, IT operations, coding, and personal assistance. A central architectural question for every such deployment is whether and how to give the agent *long-term memory*: information persisted across sessions and selectively recalled. A vibrant ecosystem of memory systems has emerged (Mem0, Letta/MemGPT-style OS memories, Zep, LangGraph checkpointing, bespoke RAG stores), and vendors compete on recall benchmarks.

Yet the evaluation of agent memory has three blind spots:

*All numbers in §5 are measured results from the pilot study (5,440 scored episodes, one model). The full preregistered study (3 models $\times$ 3 seeds $\times$ ≥ 30 arcs) is in progress; this preprint releases the benchmark, harness, and pilot findings. Passages marked [FULL RUN: ...] will be updated with the larger study. Code, traces, and preregistration: <https://github.com/smshweta/merit-bench>.

Blind spot 1: Conversational QA is not task execution. The dominant benchmarks — LoCoMo (Maharana et al., 2024), LongMemEval (Wu et al., 2025), and BEAM (Tavakoli et al., 2026) — test whether a system can answer questions about long, multi-session *conversations*. Production agents, by contrast, must use remembered facts to *choose the correct tool call*: the right customer ID, the previously agreed refund amount, the configuration fix identified during last week’s incident. Whether conversational recall scores transfer to task-execution utility is an open empirical question.

Blind spot 2: Memory is assumed to help; harm is unmeasured. Recent work notes that agents sometimes ignore retrieved memory records or use them inconsistently, and HaluMem (HaluMem, 2025) examines memory hallucination and consistency — but in conversational settings. In task execution, a stale memory (a customer’s *old* address, a rolled-back configuration) is not just unhelpful; it produces a *confidently wrong action*. No existing benchmark quantifies this failure mode under controlled corruption, nor the more basic failure our pilot surfaces: agents that demonstrably hold the correct fact in context and still do not act on it (Ignore Rate up to 0.53).

Blind spot 3: Cost is reported, but marginal utility is not. Memory systems report token counts alongside accuracy, but the decision practitioners face is economic: does adding memory raise task success enough to justify its per-task cost? We are not aware of any evaluation that reports *cost-adjusted marginal utility* — the change in success probability per marginal dollar — across memory architectures.

We address all three with MERIT. Our contributions:

1. **A task suite with controllable memory dependency and difficulty.** Episodic tool-use tasks in three domains (customer support, IT operations, personal assistant) where a tunable fraction of episodes depend on facts established in earlier episodes, and a three-tier difficulty ladder: *easy* (one fact), *medium* (multiple facts must be composed), *hard* (the fact is updated mid-arc and the latest value is required). An automated **leak check** verifies at generation time that no gold fact is present in the probe episode’s inputs or re-derivable from tools.
2. **Controlled memory corruption.** Stale, contradictory, and distractor records injected at known rates ρ , with ground-truth flags, to measure Stale-Memory Harm.
3. **Three metrics absent from prior evaluations:** Memory Utilization Rate (MUR), Ignore Rate, and Cost-Adjusted Marginal Utility (CAMU), computed automatically by the harness (§3.5).
4. **A pilot study** of 6 memory conditions $\times$ 3 domains $\times$ 3 difficulty tiers (5,440 scored episodes) with preregistered hypotheses and statistics (paired bootstrap clustered by arc; Holm–Bonferroni), demonstrating that the benchmark separates memory architectures that are indistinguishable under single-fact recall. [FULL RUN: 3 models $\times$ 3 seeds $\times$ ≥ 30 arcs]
5. **Open-source release** of the benchmark, harness, and all traces, with a deterministic \$0 mock-model mode that validates the full pipeline.

2 Related Work

2.1 Memory systems for LLM agents

MemGPT (Packer et al., 2023) frames memory as an OS-style hierarchy with the LLM paging information in and out of context. Mem0 (Chhikara et al., 2025) extracts and consolidates structured facts at write

time; Zep (Rasmussen et al., 2025) maintains a temporal knowledge graph; A-Mem (Xu et al., 2025) organizes agentic notes dynamically. Rolling summarization is the default pattern in agent frameworks such as LangChain/LangGraph. These systems papers evaluate primarily on conversational QA (LoCoMo, LongMemEval); none evaluate task-execution utility under cost accounting. MERIT’s conditions C1–C5 are deliberately *reimplementations of these architecture families* under one interface, so that architectural mechanisms (retrieval, summarization, update-on-write, hybrid) can be compared in isolation from product engineering.

2.2 Benchmarks for agent memory

LoCoMo (Maharana et al., 2024) evaluates QA over very long multi-session dialogues. LongMemEval (Wu et al., 2025) covers information extraction, multi-session reasoning, temporal reasoning, knowledge updates, and abstention over $\sim 115\text{K}$ -token histories. BEAM (Tavakoli et al., 2026) scales conversational probing to 10M tokens. HaluMem (HaluMem, 2025) evaluates hallucination at the level of memory operations (extraction, updating, QA). RealMem (Bian et al., 2026) moves toward project-oriented interaction. All are *answer-producing* evaluations; MERIT is *action-producing*: success is a predicate over the final state of a mutable world, and the knowledge-update dimension that LongMemEval and HaluMem probe conversationally becomes, in MERIT’s hard tier, a behavioral test of whether the agent *acts* on the latest value. Long-context benchmarks (RULER, Hsieh et al., 2024; BABILong, Kuratov et al., 2024) test single-pass attention over a given context, not the write-and-retrieve loop of a persistent memory.

2.3 Tool-use and agent benchmarks

τ -bench (Yao et al., 2024) evaluates tool-calling agents conversing with simulated users over domain APIs with database-state success checks — the closest environment design to MERIT’s — but each task is a single episode; memory across episodes is not exercised. AgentBench (Liu et al., 2023), SWE-bench (Jimenez et al., 2024), and WebArena (Zhou et al., 2023) similarly evaluate within-episode competence. MERIT adds the cross-episode dependency structure (arcs, plants, probes, leak check) on top of a τ -bench-style environment.

2.4 Reliability of agents

Production reports consistently rank reliability as the top barrier to agent deployment; confidently-wrong actions are costlier than abstentions. Stale-memory harm is a concrete, measurable instance: MERIT injects staleness at known rates and measures both the success drop and whether the agent re-verifies before acting.

3 The MERIT Benchmark

3.1 Design goals

G1 (Ecological validity): tasks require tool calls with programmatically verifiable end states. G2 (Control-lable dependency and difficulty): the dependent-task ratio and the difficulty tier are benchmark parameters. G3 (Adversarial realism): corruption reflects real staleness processes. G4 (Cost transparency): every token in and out of the memory system is metered. G5 (Reproducibility): deterministic seeded worlds, scripted or cached simulated users, pinned model versions, released traces, and a \$0 deterministic mock model that exercises the entire pipeline.

3.2 Environment

An **episode** is one agent session with a simulated user and a tool API over a mutable world state (SQLite). Episodes are grouped into **arcs** of 4–6 episodes sharing entities. Arcs contain **plant** episodes (a fact is

established conversationally; the user explicitly forbids acting on it yet), optional **update** episodes (the fact is revised), **probe** episodes (success requires the fact; it is absent from the episode’s inputs), and **independent** episodes (solvable within-episode).

Two safeguards proved essential in practice. The **leak check** asserts at generation time that every gold fact value is absent from the probe episode’s inputs and from the initial world state, with gold values kept unique arc-wide. **Delta scoring** marks an episode successful only if its checker predicate flips from false to true *during* that episode — in early gate runs, eager agents processed refunds during plant episodes, and probes then “succeeded” off inherited world state; delta scoring eliminates this world-state leak channel entirely (0 contaminated episodes in 5,440).

3.3 Domains

- **D1 Customer support (retail):** tools = `get_order`, `refund`, `update_address`, `get_policy`, `send_message`. Facts: agreed partial-refund amounts, updated addresses.
- **D2 IT operations:** tools = `search_logs`, `get_deploy_history`, `get_config`, `set_config`, `deploy`, `update_ticket`. Facts: diagnosed config fixes, safe rollback versions.
- **D3 Personal assistant:** tools = `get_calendar`, `create_event`, `send_email`, `get_preference`, `set_preference`. Facts: standing room/time preferences, dinner commitments (place, day, time).

3.4 Difficulty ladder and memory conditions

Difficulty: *easy* = one gold fact per probe; *medium* = multi-fact probes (every gold value must appear in the executed tool calls); *hard* = updated-fact probes: the value is planted, revised in a later episode, and the probe requires the **latest** value. The hard tier is a targeted stressor for the architectural distinction between stores that overwrite (update-on-write) and stores that accumulate (replay, retrieval).

Memory conditions, one interface (`write(episode)`, `read(context)`): C0 none; C1 full replay of prior transcripts; C2 retrieval (pilot: keyword-overlap retrieval; [FULL RUN: embedding retrieval]); C3 rolling summary (pilot: extractive truncation; [FULL RUN: LLM summarization]); C4 structured fact store with update-on-write (pilot: pattern-based extraction; [FULL RUN: LLM extraction]); C5 hybrid (C4 + C2). All conditions share the same agent scaffold (ReAct-style tool loop; Yao et al., 2023), prompts (except the memory block), tools, and decoding (temperature 0).

3.5 Metrics

- **TSR:** task success rate by programmatic end-state check, split by dependent/independent.
- **MUR:** among dependent episodes where all gold values were present in the retrieved memory block, the fraction where every gold value appears in the executed tool-call arguments (value tracing). [FULL RUN: validated by a 100-episode human audit with a second annotator; Cohen’s κ]
- **Ignore Rate** = $1 - \text{MUR}$ on episodes with correct memory present.
- **SMH:** $\text{TSR}(\text{clean}) - \text{TSR}(\text{corrupted})$ at $\rho \in \{0.1, 0.3\}$ for stale / contradiction / distractor corruption.
- **Cost & CAMU:** metered tokens and dollars per episode; $\text{CAMU} = \Delta \text{TSR}$ vs. C0 per Δcost vs. C0; break-even task value = $\Delta \text{cost} / \Delta \text{TSR}$.

Table 1: Dependent-task TSR by difficulty tier and memory condition (D1 / D2 / D3). See also Figure 1.

Tier	C1 replay	C2 retrieval	C3 summary	C4 facts	C5 hybrid
easy	1.00 / 1.00 / 0.90	0.95 / 1.00 / 0.95	0.55 / 0.65 / 0.40	1.00 / 0.60 / 0.90	0.95 / 1.00 / 1.00
medium	0.55 / 1.00 / 0.90	0.30 / 0.85 / 0.90	0.30 / 0.40 / 0.50	0.85 / 1.00 / 0.80	0.85 / 1.00 / 1.00
hard	1.00 / 1.00 / 1.00	0.60 / 0.25 / 0.20	0.00 / 0.15 / 0.00	1.00 / 0.90 / 1.00	0.70 / 0.45 / 0.75

3.6 Statistical methodology

Paired comparisons on identical task instances; 95% CIs and two-sided p by paired bootstrap (10,000 re-samples) clustered at the arc level; Holm–Bonferroni within each preregistered hypothesis family (H1–H4, committed to the public repository before experiments; commit history is the preregistration record).

4 Experimental Setup (Pilot)

Model: gpt-4.1-mini (pinned; temperature 0). Scale: 10 arcs $\times$ 5 episodes per (domain $\times$ difficulty $\times$ condition); dependent-task ratio 0.5; corruption sweep (3 modes $\times$ $\rho \in \{0.1, 0.3\}$) and LLM-paraphrased users on D1. Totals: **5,440 scored episodes, \$4.61 in API cost** ($\sim \$0.0008/\text{episode}$). Harness: Python, LiteLLM for provider-agnostic calls and metering; SQLite worlds; scripted simulated users (LLM paraphrase mode with a paraphrase-leak check on D1). The mock-model mode replays the entire grid deterministically at \$0 and is exercised by 62 unit tests, including generation determinism, leak checks, and end-to-end pipeline invariants. [FULL RUN: + frontier API model + open-weight model via vLLM, 3 seeds, ≥ 30 arcs, corruption + LLM users on all domains]

5 Results (Pilot)

5.1 Memory helps on dependent tasks; the floor is real

C0 scored **0.000** on dependent tasks in all nine (domain $\times$ difficulty) cells — the leak check holds; there is no route to the gold facts except memory. On independent tasks C0 scores 0.88–1.00, confirming task solvability. Every memory condition beats C0 on dependent tasks in every domain at easy difficulty ($\Delta\text{TSR} +0.55$ to $+1.00$; all Holm-adjusted $p \leq 0.001$, paired bootstrap clustered by arc).

5.2 The difficulty ladder dissociates architectures

Table 1 and Figure 1 show three dissociations: (1) **Updated facts break retrieval memory**: C2 falls to 0.20–0.60 on hard while C4 stays at 0.90–1.00 — retrieval surfaces stale and fresh values side by side with no recency signal, while update-on-write overwrites. (2) **Hybrid is worse than its better half** on hard (0.45–0.75 vs. C4’s 0.90–1.00): the retrieval half re-imports the staleness the fact store had eliminated. (3) **Full replay is update-immune but composition-limited**: C1 resolves recency from chronology (1.00 on hard) yet drops to 0.55 on D1-medium despite containing every fact — possessing information and composing it are different capabilities.

5.3 Agents ignore memories they hold

On the hard tier, pooling domains, the correct (latest) value was present in C2’s retrieved block in 45 probe episodes; the agent acted on it in 21 (**Ignore Rate 0.53**). Even C1/C4, whose memory blocks are clean,

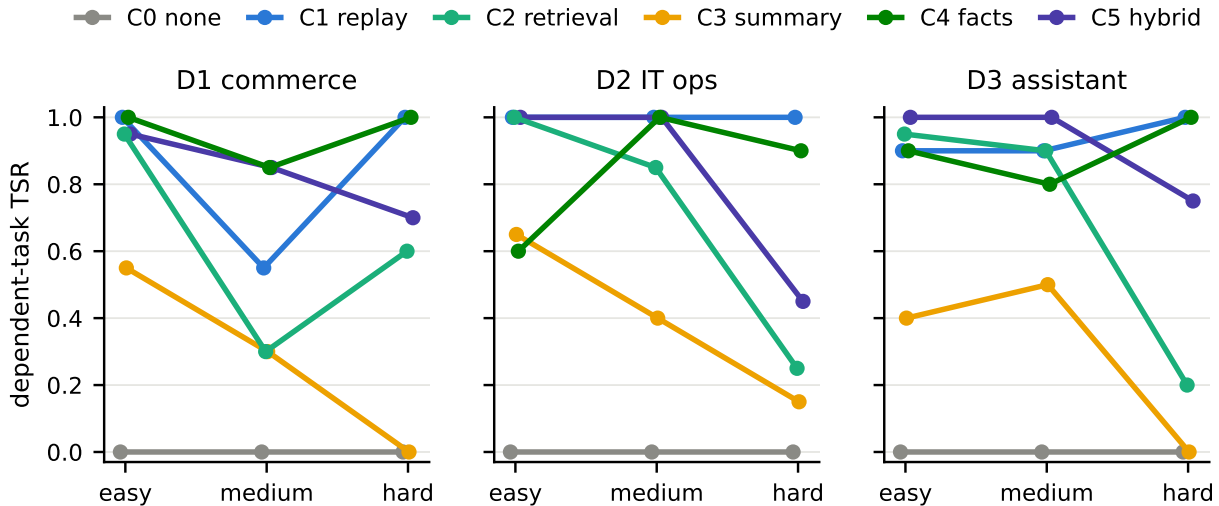


Figure 1: Dependent-task TSR across the difficulty ladder, per memory condition and domain. The hard (updated-fact) tier separates stores that overwrite (C4) or replay chronology (C1) from retrieval-based stores (C2, C5) and lossy summaries (C3).

show Ignore Rates up to 0.45 in multi-fact (medium-tier) episodes (Figure 2). This is Blind spot 2 made measurable: memory-system accuracy overstates end-task benefit unless utilization is measured.

5.4 Stale-memory harm

On D1 with stale corruption at $\rho = 0.3$ (Figure 3), TSR on dependent tasks drops by 0.05–0.25 depending on condition; the largest and only Holm-significant harms in the pilot are on the hybrid C5 (SMH +0.20 at $\rho = 0.1$, +0.25 at $\rho = 0.3$), while C4 shows the smallest harm (≤ 0.05) — consistent with update-on-write limiting the blast radius of stale records. Contradiction and distractor corruption produce smaller, mostly non-significant harms at pilot scale. [FULL RUN: corruption sweeps on all domains with verification-rate analysis]

5.5 Cost and marginal utility

Per-episode cost on D1-easy (Figure 4): C0 \$0.00046, C4 \$0.00052, C5 \$0.00076, C2 \$0.00078, C3 \$0.00088, C1 \$0.00129 (2,986 tokens/episode — $3.4\times$ C0). CAMU ranking inverts the accuracy ranking (H4 supported in all three domains): C4 delivers ~17,300 percentage points of dependent-task success per marginal dollar on D1 vs. ~1,200 for C1; in D2, C4’s marginal cost was *negative* (compact fact notes shrank prompts below the no-memory baseline) while adding +60 points. Break-even task values at pilot prices are fractions of a cent per task for all conditions — memory pays for itself at trivially low task values *when it works*; the practitioner-relevant differences are in robustness (§5.2–5.4), not raw affordability, at these model prices.

5.6 What the pilot cannot yet say

Single model, single seed, 10 arcs per cell, and starter implementations of C2/C3/C4 (keyword retrieval, truncation summaries, pattern extraction). The C3 results in particular are a floor, not an estimate, for LLM-summarization memory; C4’s pattern extractor required domain-specific patterns (an instructive brittleness — see §6). The full run upgrades all three and adds models/seeds/arcs. We report the pilot because the

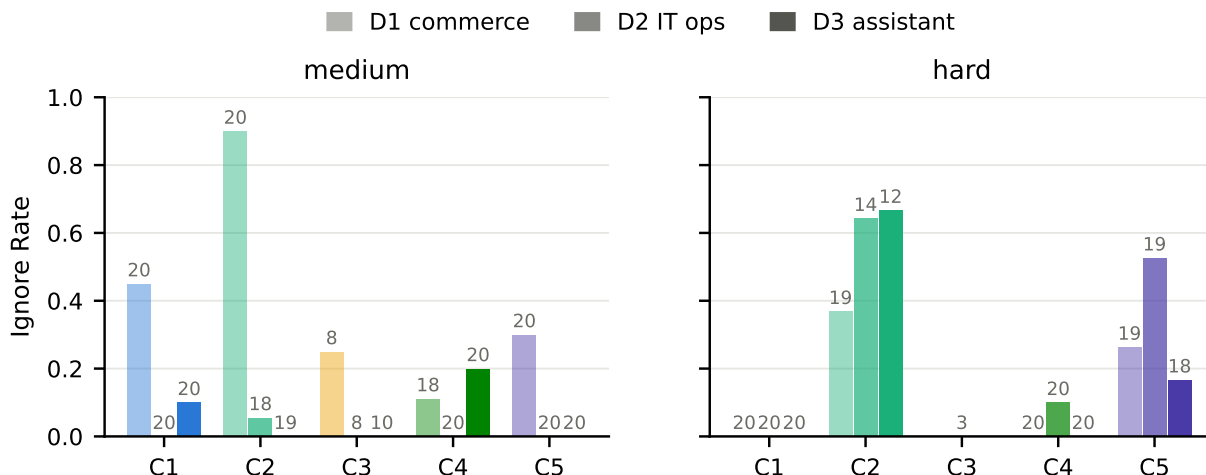


Figure 2: Ignore Rate — the fraction of dependent episodes where every gold value was present in the memory block but the agent did not act on it — by condition and domain on the medium and hard tiers. Numbers above bars are episode counts with memory present (bars at zero are shown by their count only).

dissociations in §5.2 are architectural in origin and large (0.4–0.8 absolute TSR), and because the benchmark itself — not the leaderboard — is the contribution.

6 Discussion

Practitioner guidance (provisional). If tasks depend on facts that get *revised* (addresses, configs, schedules — most operational facts), prefer update-on-write structured memory over retrieval; do not assume a hybrid inherits the better component’s behavior — measure it. Full replay is a strong accuracy baseline that fails on cost (3.4× tokens) and on multi-fact composition. Rolling summaries need real summarization; truncation is not a memory system.

Why do agents ignore correct memories? Traces show two patterns: (i) under retrieval, stale and fresh values co-occur and the agent averages, asks, or picks the stale one (no provenance/timestamps to arbitrate); (ii) under multi-fact composition, agents act on the subset of facts nearest the task phrasing. Both suggest memory *presentation* (provenance, recency marking, contradiction surfacing) matters as much as memory *storage*.

Extraction brittleness as a hidden cost. C4’s economics are the best in every domain, but its pattern-based extractor was blind outside its home domain until domain patterns were added — in production, extraction quality is the tax that structured memory pays. The full run’s LLM extractor moves this cost into the metered CAMU, where it belongs.

A methodological note. Two of our safeguards were added because early runs failed without them: eager agents leaked gold facts into world state (fixed by delta scoring), and gold-value collisions leaked across task pairs (fixed by arc-wide uniqueness). Task-execution memory benchmarks have leak channels that conversational QA benchmarks cannot express; we recommend leak checks over *world state*, not just prompts, as standard practice.

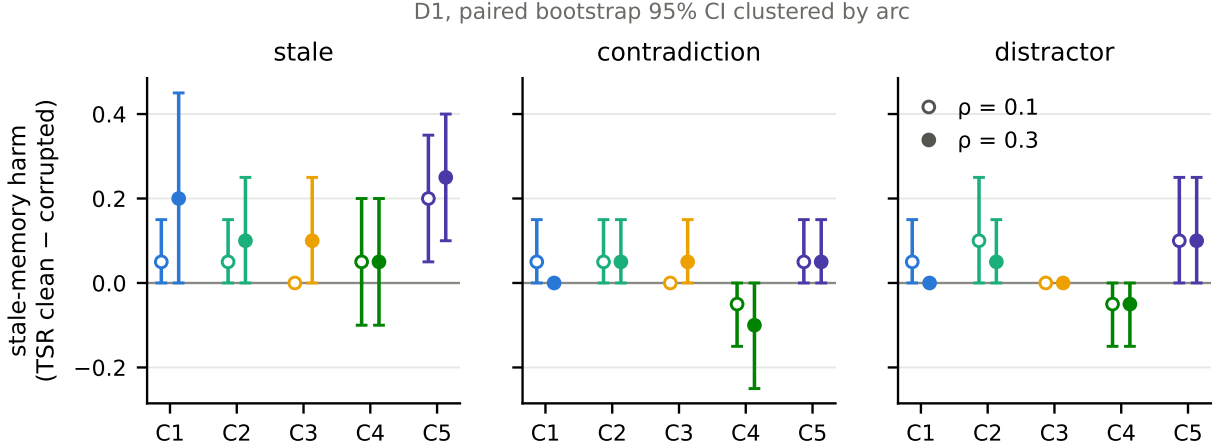


Figure 3: Stale-memory harm (TSR clean – corrupted, dependent tasks, D1) by condition, corruption mode, and corruption rate ρ ; error bars are paired bootstrap 95% CIs clustered by arc.

7 Threats to Validity

Construct: programmatic checkers may not capture all real-world success notions; the MUR tracer is string containment pending the human audit. **Internal:** prompt differences across conditions are confined to the memory block; delta scoring removes world-state carryover; the mock-model grid guards the pipeline, but mock results are never reported as findings. **External:** one model at pilot scale — architectural dissociations may shift with model strength (the full run tests whether stronger models need memory less or exploit it better); three domains; synthetic worlds with scripted users (LLM-paraphrase mode mitigates phrasing overfit on D1). **Reproducibility:** deterministic seeded generation, pinned model IDs, released traces, \$0 mock mode; API model drift is mitigated by the open-weight model in the full run.

8 Conclusion

MERIT reframes agent-memory evaluation from “can the system recall?” to “does recall change what the agent does, at what cost, and how does it fail?”. Even at pilot scale the answer is not monotone: architectures indistinguishable on single-fact recall separate by 0.4–0.8 TSR when facts must be composed or superseded, agents ignore up to half of correctly retrieved facts, and the most accurate memory is 14× less economical than the most efficient one. The benchmark, harness, traces, and preregistration are public; we invite memory-system authors to evaluate against MERIT.

References

- Bian, H., Yao, Z., Hu, S., Xu, Z., Zhang, S., Guo, Y., Yang, Z., Han, X., Wang, H., Chen, R. (2026). RealMem: Benchmarking LLMs in Real-World Memory-Driven Interaction. arXiv:2601.06966.
- Chhikara, P., Khant, D., Aryan, S., Singh, T., Yadav, D. (2025). Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. arXiv:2504.19413.
- Hsieh, C.-P., et al. (2024). RULER: What’s the Real Context Size of Your Long-Context Language Models? arXiv:2404.06654.

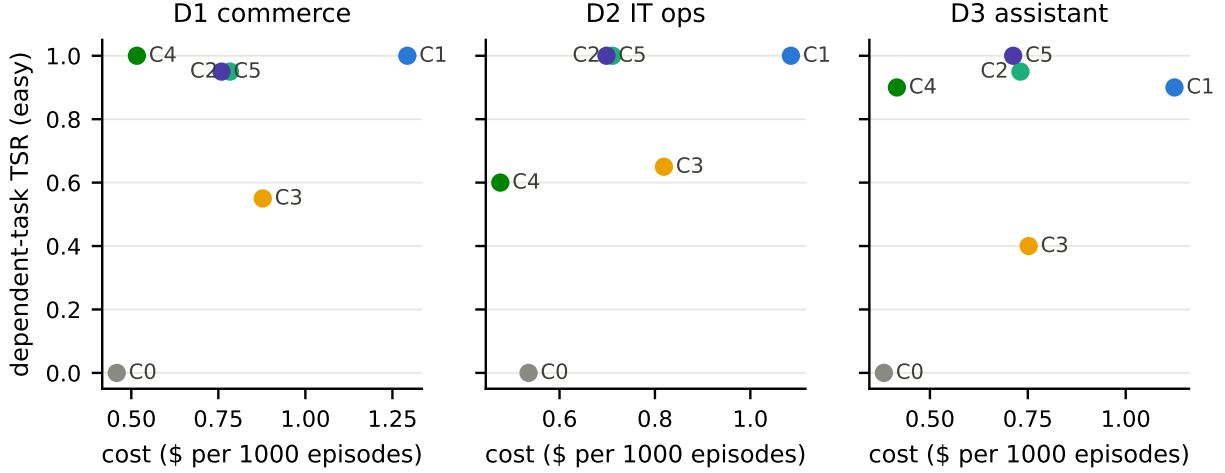


Figure 4: Metered cost per episode vs. dependent-task TSR at the easy tier. C4 sits at or near the Pareto frontier in all domains; in D2 and D3 its compact fact notes make it cheaper than several alternatives while C1 full replay pays a 2–3 $\times$ token premium for the same or lower TSR.

HaluMem (2025). HaluMem: Evaluating Hallucinations in Memory Systems of Agents. arXiv:2511.03506.

Jimenez, C., et al. (2024). SWE-bench: Can Language Models Resolve Real-World GitHub Issues? ICLR 2024. arXiv:2310.06770.

Kurатов, Y., et al. (2024). BABILong: Testing the Limits of LLMs with Long Context Reasoning-in-a-Haystack. arXiv:2402.10790.

Liu, X., et al. (2023). AgentBench: Evaluating LLMs as Agents. arXiv:2308.03688.

Maharana, A., et al. (2024). Evaluating Very Long-Term Conversational Memory of LLM Agents (LoCoMo). ACL 2024. arXiv:2402.17753.

Packer, C., et al. (2023). MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560.

Rasmussen, P., et al. (2025). Zep: A Temporal Knowledge Graph Architecture for Agent Memory. arXiv:2501.13956.

Tavakoli, M., Salemi, A., Ye, C., Abdalla, M., Zamani, H., Mitchell, J.R. (2026). Beyond a Million Tokens: Benchmarking and Enhancing Long-Term Memory in LLMs (BEAM). ICLR 2026. arXiv:2510.27246.

Wu, D., et al. (2025). LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. ICLR 2025. arXiv:2410.10813.

Xu, W., et al. (2025). A-Mem: Agentic Memory for LLM Agents. arXiv:2502.12110.

Yao, S., Shinn, N., Razavi, P., Narasimhan, K. (2024). τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. arXiv:2406.12045.

Yao, S., et al. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. ICLR 2023. arXiv:2210.03629.

Zhou, S., et al. (2023). WebArena: A Realistic Web Environment for Building Autonomous Agents. arXiv:2307.13854.